Template

Process tabular data

Process tabular data via the specified circuit.
Format the data and the corresponding output as a table.
Supports multipe input and multiple output dimensions.

Optional: For predictive networks, shift the output (predictions) to align with the input.

Requirements

```
<< "Circuits.m"
```

Circuit

```
jsonstr =

{
  "$schema": "https://creatingintelligence.org/schemas/circuit-v1.json",

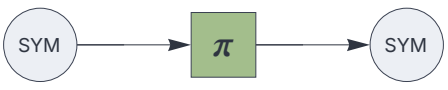
  "hyperparameters": {"default": [2000, 10]},

  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},

    {"component": "temporal", "send": [21], "receive": [11], "capacity": 2},

    {"component": "output", "plugin": "codec", "receive": [21]}
  ]
}
```

```
circ = Circuit[jsonstr];
circ["schematics"][ImageSize→240]
```



Data

```
datadir = NotebookDirectory[] <> "../../data/";
data = Import[ datadir <> "/sequences/pi-100.csv"];
 (* this file has 4 repetitions of the first 100 digits of Pi *)
```

Run

```
out = List /@ circ["function"] @@@ data;
```

Format data and output side-by-side

```
ArrayToMarkdownTable[inputs_List, out_List, predictiveQ_ : False] :=
    Module[{outputs, combinedRaw, headerRaw, allRows, colWidths, paddedTranspo

  outputs = out;

  (* align input and output rows *)
  If[ predictiveQ, outputs = RotateRight[outputs];
  outputs[[1]] = Null& /@ outputs[[1]];

  (* Convert all elements to strings *)
  combinedRaw = Map[ToString, Join[inputs, outputs, 2], {2}];

  (* Generate header row *)
  headerRaw = Join[
    ConstantArray["in", Length[First[inputs]]],
    ConstantArray["out", Length[First[outputs]]]
  ];

  allRows = Prepend[combinedRaw, headerRaw];

  (* Calculate maximum string length per column (minimum 3 for Markdown separa
  colWidths = Max[3, #] & /@ Map[Max, Map[StringLength, Transpose[allRows], {2

  (* Pad strings to match the max width of their respective columns *)
  paddedTransposed = MapThread[StringPadRight, {Transpose[allRows], colWidths}
  paddedRows = Transpose[paddedTransposed];

  (* Generate separator row with correctly sized dash strings *)
  separatorRow = StringPadRight["", #, "-"] & /@ colWidths;

  (* Format a single row with Markdown pipes *)
  formatRow[row_] := "| " <> StringRiffle[row, " | "] <> " |";

  (* Insert separator between header and body, then format and join *)
  StringRiffle[
    formatRow /@ Insert[paddedRows, separatorRow, 2],
    "\n"
  ]
]
```

```
ArrayToMarkdownTable[data, out, True]
```

in	out
3	Null
1	Null
4	Null
1	Null
5	4
9	Null
2	Null
6	Null
5	Null
3	9
5	1
8	{3, 9}
9	Null
7	2
9	Null
3	7
2	5
3	6
8	{1, 2, 5}
4	9
6	1
2	5
6	{3, 6}
4	5
3	{1, 6}
3	5
8	8
3	4
2	8
7	3
9	9
5	3
0	9
2	2
8	6
8	9
4	4

1	6
9	5
7	{2, 3, 5, 7}
1	9
6	{4, 5, 9}
9	4
3	2
9	2
9	3
3	{2, 3, 5, 7, 9}
7	2
5	{1, 9}
1	3
0	5
5	2
8	8
2	9
0	8
9	{2, 5}
7	7
4	9
9	{1, 3, 6}
4	5
4	{1, 3, 6, 9}
5	9
9	{0, 1, 3, 8, 9}
2	2
3	6
0	8
7	5
8	9
1	2
6	0
4	9
0	3
6	9
2	2
8	6
6	8
2	2
0	8
8	9

9	2
9	7
8	3
6	{4, 9}
2	2
8	6
0	6
3	2
4	3
8	0
2	9
5	0
3	{0, 1, 3, 8, 9}
4	5
2	8
1	8
1	5
7	4
0	1
6	6
7	2
3	8
1	9
4	4
1	1
5	5
9	9
2	2
6	6
5	5
3	3
5	5
8	8
9	2
7	7
9	9
3	3
2	2
3	3
8	8
4	4
6	1

2	2
6	6
4	4
3	3
3	5
8	8
3	3
2	2
7	3
9	8
5	3
0	0
2	2
8	8
8	8
4	4
1	6
9	9
7	7
1	1
6	6
9	9
3	3
9	9
9	9
3	3
7	7
5	5
1	1
0	0
5	5
8	8
2	2
0	0
9	8
7	7
4	4
9	9
4	4
4	4
5	5
9	9

2	2
3	3
0	0
7	7
8	8
1	1
6	6
4	4
0	3
6	6
2	2
8	8
6	6
2	2
0	8
8	8
9	9
9	9
8	8
6	6
2	2
8	8
0	6
3	3
4	4
8	8
2	9
5	5
3	3
4	4
2	8
1	1
1	1
7	7
0	0
6	6
7	7
3	3
1	9
4	4
1	1
5	5

9	9
2	2
6	6
5	5
3	3
5	5
8	8
9	9
7	7
9	4
3	3
2	2
3	3
8	8
4	4
6	1
2	2
6	6
4	4
3	3
3	3
8	8
3	3
2	2
7	7
9	9
5	5
0	0
2	2
8	8
8	8
4	4
1	1
9	9
7	7
1	1
6	6
9	9
3	3
9	9
9	9
3	3

7	7
5	5
1	1
0	0
5	5
8	8
2	2
0	0
9	9
7	7
4	4
9	9
4	4
4	4
5	5
9	9
2	2
3	3
0	0
7	7
8	8
1	1
6	6
4	4
0	0
6	6
2	2
8	8
6	6
2	2
0	0
8	8
9	9
9	9
8	8
6	6
2	2
8	0
0	8
3	3
4	4
8	8

2	2
5	5
3	3
4	4
2	2
1	1
1	1
7	7
0	0
6	6
7	7
3	3
1	1
4	4
1	1
5	5
9	9
2	2
6	6
5	5
3	3
5	5
8	8
9	9
7	7
9	9
3	3
2	2
3	3
8	8
4	4
6	6
2	2
6	6
4	4
3	3
3	3
8	8
3	3
2	2
7	7
9	9

5	5
0	0
2	2
8	8
8	8
4	4
1	1
9	9
7	7
1	1
6	6
9	9
3	3
9	9
9	9
3	3
7	7
5	5
1	1
0	0
5	5
8	8
2	2
0	0
9	9
7	7
4	4
9	9
4	4
4	4
5	5
9	9
2	2
3	6
0	0
7	7
8	8
1	1
6	6
4	4
0	0
6	6

2	2
8	8
6	0
2	2
0	0
8	8
9	9
9	9
8	8
6	6
2	2
8	8
0	0
3	3
4	4
8	8
2	2
5	5
3	3
4	4
2	2
1	1
1	1
7	7
0	0
6	6
7	7